SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:
"Artificial intelligence is"
the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:
"The cat sits on the mat."
and
"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- **Unigram model**
- **Bigram model**
- **Trigram model**
- **Smoothed model**

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 – N-gram Based Next-Word Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities. For an incomplete sentence such as: “Artificial intelligence is” the system should identify and display the top-5 most probable next words. The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Aim

To design and implement a Python-based N-gram language model that predicts the next word in an incomplete sentence using unigram, bigram, and trigram probabilities.

Objective

• Preprocess an English corpus.

• Tokenize the corpus.

• Construct unigram, bigram, and trigram models.

• Calculate frequency counts.

• Calculate maximum-likelihood probabilities.

• Predict the top-5 next words.

• Demonstrate the zero-probability problem for unseen N-grams.

• Calculate prediction accuracy.

Algorithm

• Store an English training corpus.

• Convert text to lowercase.

• Tokenize the corpus into words.

• Generate unigram, bigram, and trigram counts.

• Calculate unigram, bigram, and trigram probabilities.

• Accept an incomplete sentence.

• Select N = 1, 2, or 3.

• Find candidate next words.

• Rank candidates according to probability.

Python Program

```
import re
from collections import Counter

corpus = """
artificial intelligence is changing the world artificial
intelligence is used in healthcare artificial intelligence is
used in education machine learning is a part of artificial
intelligence machine learning can solve complex
problems machine learning can analyze large datasets
deep learning is a branch of machine learning deep
learning can recognize patterns data science is
important for business data science can provide useful
insights
natural language processing helps computers understand language
natural language processing is used in chatbots computers can
process large amounts of data students learn machine learning
students learn artificial intelligence students can build intelligent
applications intelligent applications can solve real world problems
"""

def tokenize(text):     return re.findall(r'\b[a-z]+\b',
text.lower())
```

```
words = tokenize(corpus)

unigram = Counter(words) bigram =
Counter(zip(words, words[1:])) trigram =
Counter(zip(words, words[1:], words[2:]))

total_words = len(words)

def unigram_probability(word):
   return unigram[word] / total_words

def bigram_probability(w1, w2):     return bigram[(w1, w2)] /
unigram[w1] if unigram[w1] else 0

def trigram_probability(w1, w2, w3):     return trigram[(w1, w2, w3)] /
bigram[(w1, w2)] if bigram[(w1, w2)] else 0

def predict(sentence, n):
   tokens = tokenize(sentence)
candidates = {}

   if n == 1:         for word in unigram:
candidates[word] = unigram_probability(word)

   elif n == 2:         if
not tokens:
return []

      last_word = tokens[-1]

      for (w1, w2), count in bigram.items():            if w1 ==
last_word:                candidates[w2] =
bigram_probability(w1, w2)

   elif n == 3:         if
len(tokens) < 2:
return []

      w1, w2 = tokens[-2], tokens[-1]
```

```
    for (a, b, c), count in trigram.items():            if a == w1
and b == w2:                candidates[c] = trigram_probability(a,
b, c)

  return sorted(candidates.items(), key=lambda x: x[1], reverse=True)[:5]

print("N-GRAM LANGUAGE MODEL") print("----
----------------")

print("\nUnigram Counts:")
print(unigram)

print("\nBigram Counts:") print(bigram)

print("\nTrigram Counts:") print(trigram)

sentence = input("\nEnter incomplete sentence: ") n
= int(input("Enter N (1, 2 or 3): "))

predictions = predict(sentence, n)

print("\nTop-5 Next Word Predictions:")

if predictions:     for word, probability in predictions:
print(f"{word:15} {probability:.4f}") else:     print("No
prediction available. The N-gram is unseen.")

print("\nTesting Multiple Sentences")

test_data = [     ("artificial intelligence is",
"changing"),
  ("machine learning can", "solve"),

  ("deep learning can", "recognize"),
("data science is", "important") ]

correct = 0
```

```
for sentence, actual in test_data:     result
= predict(sentence, 3)
    predicted_words = [word for word, probability in result]

    if actual in predicted_words:
correct += 1

accuracy = (correct / len(test_data)) * 100
print("\nPrediction Accuracy:", accuracy, "%")
```

Advantages

• Simple to implement.

• Easy to understand.

• Fast prediction for small corpora.

• Useful for basic autocomplete systems.

Limitations

• Unseen N-grams receive probability zero.

• Requires large amounts of training data.

• Cannot understand the actual meaning of words.

• Long-distance relationships between words are ignored.

• Performance decreases for sparse data.

Conclusion

Thus, an N-gram language model was successfully implemented using unigram, bigram, and trigram models. The system calculated word probabilities and predicted the top-5 possible next words. The experiment also demonstrated the zero-probability problem of an unsmoothed N-gram model.

Q2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as: "Machine learning can". When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights. The system should generate the top-5 next-word predictions using: • Unsmoothed N-gram model • Backoff model • Deleted Interpolation model Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Aim

To implement an enhanced N-gram language prediction system using backoff and interpolation to overcome the zero-probability problem of an unsmoothed N-gram model.

Objective

• Unsmoothed N-gram model

• Backoff model

• Deleted/interpolated model

Algorithm

For backoff, use the sequence Trigram → Bigram → Unigram. If the trigram does not exist, use the bigram; if the bigram does not exist, use the unigram.

For interpolation, combine all three probabilities using $\lambda1 + \lambda2 + \lambda3 = 1$. Example weights are $\lambda1 = 0.2$, $\lambda2 = 0.3$, and $\lambda3 = 0.5$.

$P(w■|w■■■,w■■■) = \lambda1P(w■) + \lambda2P(w■|w■■■) + \lambda3P(w■|w■■■,w■■■)$

Python Program

```
import re
```

```
from collections import Counter

corpus = """
machine learning can solve complex problems machine
learning can analyze large datasets machine learning is
a part of artificial intelligence artificial intelligence can
solve real world problems artificial intelligence is
changing the world deep learning can recognize
patterns deep learning is a branch of machine learning
data science can provide useful insights data science is
important for business

natural language processing can understand language
natural language processing is used in chatbots
computers can process large amounts of data students
learn machine learning students can build intelligent
applications """

def tokenize(text):     return re.findall(r'\b[a-z]+\b',
text.lower())

words = tokenize(corpus)

unigram = Counter(words) bigram =
Counter(zip(words, words[1:])) trigram =
Counter(zip(words, words[1:], words[2:]))

total = len(words)

def p_unigram(word):
    return unigram[word] / total

def p_bigram(w1, w2):
if unigram[w1] == 0:
        return 0
    return bigram[(w1, w2)] / unigram[w1]
```

```
def p_trigram(w1, w2, w3):
if bigram[(w1, w2)] == 0:
        return 0
    return trigram[(w1, w2, w3)] / bigram[(w1, w2)]

def unsmoothed_probability(w1, w2, word):
return p_trigram(w1, w2, word)

def backoff_probability(w1, w2, word):
tri = p_trigram(w1, w2, word)

    if tri > 0:
return tri

    bi = p_bigram(w2, word)

    if bi > 0:
return bi

    return p_unigram(word)

lambda1 = 0.2
lambda2 = 0.3
lambda3 = 0.5

def interpolation_probability(w1, w2, word):
    return (         lambda1 *
p_unigram(word)         + lambda2 *
p_bigram(w2, word)
        + lambda3 * p_trigram(w1, w2, word)

    )

def predict(sentence, method):
tokens = tokenize(sentence)

    if len(tokens) < 2:
return []
```

```
    w1, w2 = tokens[-2], tokens[-1]     results
= {}

    for word in unigram:         if
method == "unsmoothed":
            probability = unsmoothed_probability(w1, w2, word)         elif
method == "backoff":
            probability = backoff_probability(w1, w2, word)         else:
probability = interpolation_probability(w1, w2, word)

        results[word] = probability

    return sorted(
results.items(),
key=lambda x: x[1],
reverse=True     )[:5]

sentence = input("Enter incomplete sentence: ")

print("\nUnsmoothed N-gram Predictions:")
print(predict(sentence, "unsmoothed"))

print("\nBackoff Predictions:")
print(predict(sentence, "backoff"))

print("\nInterpolation Predictions:")
print(predict(sentence, "interpolation"))
```

Comparison

Model	Zero Probability	Coverage	Accuracy
Unsmoothed	High	Low	Lower
Backoff	Low	High	Better
Interpolation	Very Low	Very High	Usually better

Conclusion

The experiment demonstrated that an unsmoothed N-gram model suffers from zero probabilities when an N-gram is not present in the training corpus. The backoff model solves this problem by using lower-order models, while interpolation combines probabilities from unigram, bigram, and trigram models. Therefore, backoff and interpolation are more suitable for sparse language data.

Q3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty. For example, compare sentences such as: “The cat sits on the mat.” and “The quantum processor redesigned the...” The system should evaluate how predictable the next word is based on the trained language model. The program should also compare entropy values obtained using: • Unigram model • Bigram model • Trigram model • Smoothed model Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Aim

To implement unigram, bigram, trigram, and smoothed language models and evaluate their predictive uncertainty using entropy.

Formula

Entropy: H = −(1/N) Σ log■ P(w■ | context). Low entropy means the sentence is relatively predictable; high entropy means the sentence is difficult for the model to predict.

Algorithm

• Create training corpus.

• Create testing corpus.

• Tokenize both corpora.

• Construct unigram, bigram, and trigram models.

• Calculate probabilities for test words.

• Calculate entropy.

• Apply smoothing for unseen words.

• Compare entropy values.

• Classify sentences as high or low uncertainty.

Python Program

```
import re
import math
from collections import Counter

training_text = """ the
cat sits on the mat the
cat eats the food the dog
sits on the mat the dog
eats the food the boy
plays with the ball the
girl plays with the doll
the student reads a book
the student learns
machine learning
machine learning is
useful artificial
intelligence is useful the
computer processes data
the computer learns
patterns """

test_sentences = [     "the cat
sits on the mat",
    "the dog eats the food",     "the quantum
processor redesigned the" ]
```

```
def tokenize(text):     return re.findall(r'\b[a-z]+\b',
text.lower())

words = tokenize(training_text)

unigram = Counter(words) bigram =
Counter(zip(words, words[1:])) trigram =
Counter(zip(words, words[1:], words[2:]))

V = len(unigram)
N = len(words)

def unigram_prob(word, smooth=False):
if smooth:
      return (unigram[word] + 1) / (N + V)     return
unigram[word] / N if unigram[word] else 0

def bigram_prob(w1, w2, smooth=False):
numerator = bigram[(w1, w2)]
denominator = unigram[w1]

   if smooth:        return (numerator + 1) /
(denominator + V)

   if denominator == 0:
return 0

   return numerator / denominator

def trigram_prob(w1, w2, w3, smooth=False):
numerator = trigram[(w1, w2, w3)]     denominator
= bigram[(w1, w2)]

   if smooth:        return (numerator + 1) /
(denominator + V)

   if denominator == 0:
      return 0
```

```
    return numerator / denominator

def entropy(sentence, model, smooth=False):
    tokens = tokenize(sentence)     probabilities
= []

    for i, word in enumerate(tokens):

        if model == "unigram":
            p = unigram_prob(word, smooth)

        elif model == "bigram":
if i == 0:
                p = unigram_prob(word, smooth)
else:
                p = bigram_prob(tokens[i-1], word, smooth)

        else:             if i
< 2:
                p = unigram_prob(word, smooth)
else:
                p = trigram_prob(
tokens[i-2],                    tokens[i-
1],                  word,
smooth                )

        if p > 0:
probabilities.append(p)

    if not probabilities:
return float("inf")

    return -sum(math.log2(p) for p in probabilities) / len(probabilities)

for sentence in test_sentences:
print("\nSentence:", sentence)
    for model in ["unigram", "bigram", "trigram"]:
```

```
        value = entropy(sentence, model)

        if value == float("inf"):
            print(model, ": Undefined / unseen")        else:
    print(model, "Entropy:", round(value, 4))

    smoothed = entropy(sentence, "trigram", True)

    print("Smoothed Trigram Entropy:", round(smoothed, 4))

    if smoothed > 5:
        print("Classification: High Predictive Uncertainty")     else:

        print("Classification: Low Predictive Uncertainty")
```

Interpretation

“The cat sits on the mat.” contains word sequences similar to those in the training corpus, so it is expected to have lower entropy and higher predictability.

“The quantum processor redesigned the...” contains information that is absent or rare in the training corpus, so it is expected to have higher entropy and lower predictability.

Comparison

Model	Context Used	Expected Entropy
Unigram	One word	Higher
Bigram	Previous word	Lower
Trigram	Previous two words	Usually lower
Smoothed	Context + unseen words	More reliable

Conclusion

The entropy experiment demonstrated that entropy can measure the uncertainty of language prediction. Sentences containing familiar word sequences generally have lower entropy, while unfamiliar or unseen sequences produce higher entropy. Smoothed models provide more reliable probability estimates because they avoid zero probabilities.

Q4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences. The system should implement three approaches: 1. Rule-Based POS Tagging – Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags. 2. Stochastic POS Tagging – Construct a statistical POS tagger using word/tag frequencies, tag transition probabilities, and probabilities obtained from a training corpus. 3. Transformation-Based Tagging – Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors. For example: “I book a ticket.” and “I read a book.” The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Aim

To implement and compare rule-based, stochastic, and transformation-based POS tagging approaches for English sentences.

POS Tags

Tag	Meaning	Example
NN	Noun	book
NNS	Plural noun	books
VB	Verb	read
VBD	Past tense verb	read
VBG	Gerund	reading
VBN	Past participle	written
JJ	Adjective	good
RB	Adverb	quickly
DT	Determiner	the

PRP	Personal pronoun	I
IN	Preposition	in
CC	Conjunction	and

1. Rule-Based POS Tagging

Rules are manually defined. For example: I → PRP, the → DT, a → DT, book → NN, quickly → RB.

Suffix rules can also be used, such as -ing → VBG, -ly → RB, -ed → VBD, and -ous → JJ.

2. Stochastic POS Tagging

A statistical model calculates P(tag|word) and considers tag transition probabilities P(tag■|tag■■■). The most probable tag sequence is selected.

3. Transformation-Based Tagging

The system initially assigns basic tags and then applies correction rules. For example, “I book a ticket.” gives book → VB, while “I read a book.” gives book → NN. This demonstrates why context is important.

Python Program

Install NLTK using: pip install nltk

```
import nltk
from collections import Counter, defaultdict from
nltk.corpus import brown

nltk.download('brown')
nltk.download('universal_tagset')

tagged_words = brown.tagged_words(tagset='universal')

word_tag_counts = defaultdict(Counter)
tag_transitions = defaultdict(Counter) tag_counts
= Counter()

previous_tag = "<START>"
```

```
for word, tag in tagged_words:
    word = word.lower()
word_tag_counts[word][tag] += 1
tag_counts[tag] += 1
    tag_transitions[previous_tag][tag] += 1
previous_tag = tag

def stochastic_tag(sentence):
    words = sentence.lower().split()
result = []     previous = "<START>"

    for word in words:          if word in
word_tag_counts:
            possible_tags = word_tag_counts[word]
best_tag = None             best_score = -1

            for tag in possible_tags:
word_probability = (
word_tag_counts[word][tag]                    /
sum(possible_tags.values())                 )

                transition_probability = (
tag_transitions[previous][tag]                    /
tag_counts[previous]                    if
tag_counts[previous] > 0                    else 0
                )

                score = word_probability * transition_probability

                if score > best_score:
best_score = score                    best_tag = tag

            result.append((word, best_tag))
previous = best_tag         else:
            result.append((word, "NOUN"))
previous = "NOUN"
```

```
    return result

def rule_based_tag(sentence):
    words = sentence.lower().split()     result
= []

    dictionary = {         "i": "PRON", "you": "PRON", "he":
"PRON", "she": "PRON",
        "it": "PRON", "we": "PRON", "they": "PRON",
        "the": "DET", "a": "DET", "an": "DET",
        "is": "VERB", "am": "VERB", "are": "VERB",
        "was": "VERB", "were": "VERB",
        "book": "NOUN", "ticket": "NOUN",
        "cat": "NOUN", "dog": "NOUN",
"read": "VERB", "plays": "VERB"     }

    for word in words:         if word
in dictionary:             tag =
dictionary[word]         elif
word.endswith("ing"):
            tag = "VERB"         elif
word.endswith("ly"):
            tag = "ADV"         elif
word.endswith("ed"):
            tag = "VERB"         elif
word.endswith("ous"):
            tag = "ADJ"
else:             tag =
"NOUN"

        result.append((word, tag))

    return result
```

```
def transformation_based_tag(sentence):
result = rule_based_tag(sentence)     words
= sentence.lower().split()

    for i, (word, tag) in enumerate(result):

        if i > 0 and words[i-1] in ["a", "an", "the"]:
            result[i] = (word, "NOUN")

        if i > 0 and words[i-1] == "i":            if word in ["book",
"read", "play", "write", "like"]:               result[i] = (word, "VERB")

    return result

sentence = input("Enter an English sentence: ")

print("\nRule-Based POS Tagging:")
print(rule_based_tag(sentence))

print("\nStochastic POS Tagging:")
print(stochastic_tag(sentence))

print("\nTransformation-Based POS Tagging:")
print(transformation_based_tag(sentence))
```

Example 1

Input: “I book a ticket”

Rule-based tagging may initially assign book as a noun. Transformation-based tagging can correct it to a verb because it follows “I”.

Example 2

Input: “I read a book”

Expected contextual tagging: I → PRON, read → VERB, a → DET, book → NOUN.

This demonstrates that the same word can have different POS tags depending on its context.

Evaluation Measure

POS tagging accuracy = (Correctly Tagged Words / Total Words) × 100. For example, if 92 out of 100 words are correctly tagged, the accuracy is 92%.

Comparison

Feature	Rule-Based	Stochastic	Transformation-Based
Knowledge	Hand-written rules	Training corpus	Initial tags + rules
Context	Limited	Strong	Moderate/strong
Training data	Not required	Required	Usually required
Ambiguity handling	Limited	Good	Good
Implementation	Simple	More complex	Moderate
Accuracy	Depends on rules	Usually high	Usually high
Adaptability	Low	High	Moderate

Advantages and Limitations

Rule-Based

• Easy to understand.

• Does not require large training data.

• Rules can be manually controlled.

• Difficult to create rules for every language situation.

• Poor handling of ambiguity.

• Rules become complex for large systems.

Stochastic

• Uses actual corpus statistics.

• Handles ambiguity better.

• Can learn language patterns.

• Requires tagged training data..

Transformation-Based

• Combines simple tagging with correction rules.

• Can correct common tagging mistakes.

• Easier to interpret than some statistical models.

• Requires carefully designed transformation rules.

• May not handle all contexts.

• Rule ordering can affect results.

Conclusion

A comparative POS tagging system was successfully designed using rule-based, stochastic, and transformation-based approaches. The experiment demonstrated that POS tagging depends strongly on context. The word “book”, for example, can be a noun in “I read a book” and a verb in “I book a ticket.” Among the three approaches, stochastic tagging generally provides better contextual handling when sufficient training data is available, while transformation-based tagging can improve initial tagging through correction rules.